

プログラミング入門2

第6回

ポインタ

篠埜 功

今日の内容

- ポインタ

アドレス

一般にCなどの命令型言語では、変数というのは直接的にはアドレスを表す。間接的に、そのアドレスの中身の値を表している。中身の値は代入によって変化し得る。(変数の値は中身の値である。)

変数だけでなく、配列、配列の各要素もアドレスを持つ。

アドレス

```
int main (void) {  
    char x = 10;  
    x = x + 1;  
    return 0;  
}
```

100

101

102

103

104

10

例えば、変数x用の領域が101番地だったとする。
そのとき、101が、xのアドレスである。また、初期状態
では10が式xの値である。変数xの値は代入式によって
11に変わる。

アドレス

```
int main (void) {  
    char x[3] = {0};  
    x [2] = 1;  
    return 0;  
}
```

100	
101	0
102	0
103	0
104	

配列x用の領域が101から103番地だったとする。そのとき、式x[0]のアドレスは101, 式x[1]のアドレスは102, 式x[2]のアドレスは103である。また、初期状態では式x[0], x[1], x[2]の値は0、代入後はx[2]の値は1である。

&演算子

変数や配列の要素のアドレスを取得する演算子が&演算子である。例えば、

```
int x;
```

という宣言の下で、&をxに適用することによって、変数xの領域のアドレスを取得できる。

&演算子をアドレス演算子とも言う。

(他の例)

```
int a [5];
```

という宣言下で、&をa[2]に適用すると、配列aの2番目(0から数えて)の領域のアドレスが得られる。

例(打ち込んで確認)

変数のアドレスを表示

```
#include <stdio.h>
int main (void) {
    int x;
    double y;
    printf ("The address of x is %p.\n", &x);
    printf ("The address of y is %p.\n", &y);
    return 0;
}
```

printfの変換指定子には%pを用いる。

(注意)プログラムの実行毎に、また、関数呼び出しの系列によって、変数用の領域(アドレス)は変わる。

例

配列の要素のアドレスを表示

```
#include <stdio.h>
int main (void) {
    int a [5];
    printf ("The address of a[0] is %p.\n", &a[0]);
    printf ("The address of a[1] is %p.\n", &a[1]);
    printf ("The address of a[2] is %p.\n", &a[2]);
    printf ("The address of a[3] is %p.\n", &a[3]);
    printf ("The address of a[4] is %p.\n", &a[4]);
    return 0;
}
```

ポインタ

&演算子の適用対象の式の型が t 型るとき、式 $\&e$ の型は $t * \text{型}$ (型 t へのポインタ型と読む)である。また、式 $\&e$ の値(アドレス)を、通常、「 e へのポインタ」と表現する。

例えば、

```
int y;
```

という宣言下で、 $\&y$ という式の型は $\text{int} * \text{型}$ である。また、式 $\&y$ の値は、変数 y へのポインタである。

ポインタの型によって、ポインタが指している先の型情報が分かる(のでコンパイル時に型に関する整合性の検査ができる)。また、ポインタに対する足し算、引き算(後述)の意味が型によって異なる。

アドレス演算子 &

アドレス演算子適用式の構文

&式

アドレス演算子適用式 $\&e$ の意味

式 $\&e$ の評価結果は式 e のアドレスである。式 e がアドレスを持たない式の場合、コンパイル時にエラーとなる。

アドレス演算子適用式 $\&e$ の型

式 e の型が t 型のとき、式 $\&e$ の型は t^* 型である。

ポインタ型の変数の宣言

ポインタ型の変数を宣言することができる。
 t *型の変数の宣言は以下の形で行う。

t *変数名;

例えば、int型へのポインタ型の変数の宣言は、
int *x;
のように行う。

注意事項

ポインタ型の変数を複数個宣言する場合、

```
int *a, *b;
```

のように、変数名の前に全部*をつける必要がある。

```
int *a, b;
```

と宣言すると、aはint型へのポインタ型の変数になるが、bはint型の変数となる。

例

```
#include <stdio.h>
int main (void) {
    int x;
    double y;
    int *px;
    double *py;
    px = &x;
    py = &y;
    printf ("The address of x is %p.¥n", px);
    printf ("The address of y is %p.¥n", py);
    return 0;
}
```

間接演算子 *

あるアドレスに格納されている値を取り出したいとする。そのとき、間接演算子*を用いる。

例えば、

```
int x = 5;
```

```
int *p;
```

```
p = &x;
```

という状況で、式*pは、5という値を持つ。

式*pは、pが変数xへのポインタの場合は、xの別名である。(xと置き換えても同じ意味。)

ただし、pの値を変えると(代入によって変更可能、後述)、xと同じ意味ではなくなる。

例(打ち込んで確認)

```
#include <stdio.h>
int main (void) {
    int x = 5;
    double y = 5.5;
    int *px;
    double *py;
    px = &x;
    py = &y;
    printf ("The value of x is %d.¥n", *px);
    printf ("The value of y is %f.¥n", *py);
    return 0;
}
```

配列とポインタ

配列aが、`int a [5];`で宣言されているとする。

配列 a	a[0]	100番地
	a[1]	104番地
	a[2]	108番地
	a[3]	112番地
	a[4]	116番地

a [0]の領域のアドレスが100番地から始まる場合、
a[1]は104番地、a[2]は108番地、a[3]は112番地、
a[4]は116番地から始まる。(int型が4byteの場合)

配列とポインタ(2)

配列 a	a[0]	100番地
	a[1]	104番地
	a[2]	108番地
	a[3]	112番地
	a[4]	116番地

a [0]の領域が100番地の場合、&a[0]の値は100である。この状況において、

```
int *p;
```

で宣言された変数pに対して以下の代入を実行すると、pの値は100になる。

```
p = &a[0];
```

配列とポインタ(3)

配列 a	a[0]	100番地
	a[1]	104番地
	a[2]	108番地
	a[3]	112番地
	a[4]	116番地

```
int *p;
```

で宣言された変数pに対して

```
p = &a[0];
```

を実行すると、&a[0]が100の場合、pには100が代入される。その状況で、式 $p + 1$ の値は104である(int型が4byteの場合)。

C言語の規格で、 $p + 1$ は、pが指す配列の要素の次の要素を指すということが定められている(次ページ参照)。

ポインタ型の式とint型の式の 足し算（重要）

ポインタ型の式とint型の式は足し算を行うことができる。

t^* 型の式 e_1 と int 型の式 e_2 の足し算式の意味

e_1 が配列 a の i 番目の要素を指すポインタで、
 e_2 の値が n のとき、 $e_1 + e_2$ あるいは $e_2 + e_1$ は、
配列 a の $(i + n)$ 番目の要素を指すポインタである。

注意

ポインタ型の式 e_1 とint型の式 e_2 の足し算は、配列の範囲を超えないように注意する必要がある。足し算の結果が配列の範囲を超える場合、(配列の最後の要素+ 1番目を除いて)足し算の意味は未定義である。

ポインタの足し算の結果が配列の最後の要素+1番目を指すのは良いが、指す先の中身を*で取得してはいけない。

(背景にある考え方)ポインタの値を配列の先頭を指すところから順に1ずつ足して行って配列の最後までループで処理するような場合に、終了条件を満たさなくなるのはポインタが配列の最後の要素+1番目を指すときなので、この場合を許している。

参考

```
#include <stdio.h>
int main (void) {
    int a[10];
    int *p;
    for(p=&a[0]; p<&a[0]+10; p=p+1)
        *p = 2;
    return 0;
}
```

これはa[0]からa[9]に2を代入するプログラム。後で説明するが、&a[0]はaと書いてもよい。

上記のプログラムにおいて、a+10は配列の最後の要素の1つ隣りなのでISOの規格では許される。ただし、a+10が指している場所の値を使ってはいけない。つまり、a[10]の値を使ってはいけない。

ポインタ型の式とint型の式の 引き算

ポインタ型の式とint型の式は引き算を行うことができる(意味は足し算の場合と同様)。

ポインタ型の式同士の引き算もできる(ポインタ型の式同士の足し算はできないが)。

例(打ち込んで確認)

```
#include <stdio.h>
```

```
int main (void) {
```

```
    int a [5] = {10,20,30,40,50};
```

```
    int *p;
```

```
    p = &a[0];
```

```
    printf ("The value of a[0] is %d.\n", *p);
```

a[0]の別名

```
    printf ("The value of a[1] is %d.\n", *(p+1));
```

a[1]の別名

```
    printf ("The value of a[2] is %d.\n", *(p+2));
```

a[2]の別名

```
    printf ("The value of a[3] is %d.\n", *(p+3));
```

a[3]の別名

```
    printf ("The value of a[4] is %d.\n", *(p+4));
```

a[4]の別名

```
    return 0;
```

```
}
```

ポインタ型変数の値の変更

ポインタ型変数の値を代入によって変更できる。

```
int *p;
```

で宣言された変数pに対して

```
p = &a[0];
```

を実行すると、&a[0]が100の場合、pには100が代入される。この状況ではpはa[0]へのポインタである。この状況下で

```
p = p + 1;
```

が実行されると、pには104が代入される。

(ISO規格により、p=&a[0]; p=p+1; を実行すると、pはa[1]を指すようになる。つまりpはa[1]へのポインタになる。)

例

```
#include <stdio.h>
```

```
int main (void) {
```

```
    int a[5] = {10,20,30,40,50};
```

```
    int *p;
```

```
    p = &a[0];
```

```
    printf ("The value of a[0] is %d.\n", *p);
```

a[0]の別名

```
    p = p+1;
```

```
    printf ("The value of a[1] is %d.\n", *p);
```

a[1]の別名

```
    p = p+1;
```

```
    printf ("The value of a[2] is %d.\n", *p);
```

a[2]の別名

```
    p = p+1;
```

```
    printf ("The value of a[3] is %d.\n", *p);
```

a[3]の別名

```
    p = p+1;
```

```
    printf ("The value of a[4] is %d.\n", *p);
```

a[4]の別名

```
    return 0;
```

```
}
```

例：配列の要素の和の計算 (打ち込んで確認)

```
#include <stdio.h>
int main (void) {
    int a[5] = {10,20,30,40,50};
    int *p;
    int sum=0, i;
    p = &a[0];
    for (i=0; i<5; i=i+1)
        sum = sum + *(p + i);
    printf ("sum = %d¥n", sum);
    return 0;
}
```

a[i]の別名

配列を関数に渡したい場合

配列は関数には渡せない。

配列の先頭要素へのポインタを関数に渡す。

例(打ち込んで確認)

```
#include <stdio.h>
int sum (int *p) {
    int sum=0, i;
    for (i=0; i<5; i=i+1)
        sum = sum + *(p + i);
    return sum;
}
int main (void) {
    int a[5] = {10,20,30,40,50};
    printf ("sum = %d¥n", sum (&a[0]));
    return 0;
}
```

main関数中のa[i]の別名

配列の長さ

関数に配列の先頭要素へのポインタだけを渡すと、配列の長さの情報は呼ばれた関数側では分からない。配列を受け取る関数に長さ情報も渡したい場合は、int型の引数で長さ情報を渡す。

例

```
#include <stdio.h>
int sum (int *p, int size) {
    int sum=0, i;
    for (i=0; i<size; i=i+1)
        sum = sum + *(p + i);
    return sum;
}
int main (void) {
    int a[5] = {10,20,30,40,50};
    printf ("sum = %d¥n", sum (&a[0], 5));
    return 0;
}
```

main関数中のa[i]の別名

[] の意味

C言語では、[] の意味はポインタを使って定義されている。

式 $e_1 [e_2]$ は、 $*(e_1 + e_2)$ のsyntax sugarである。

Syntax sugarとは、ある構文の別の書き方という意味である。
 $e_1 [e_2]$ の形の式はコンパイル時に $*(e_1 + e_2)$ に変換されてから処理される。

例

```
#include <stdio.h>
int sum (int *p, int size) {
    int sum=0, i;
    for (i=0; i<size; i=i+1)
        sum = sum + p[i];
    return sum;
}

int main (void) {
    int a[5] = {10,20,30,40,50};
    printf ("sum = %d¥n", sum (&a[0], 5));
    return 0;
}
```

p[i]は*(p+i)と同じである。

ポインタ型の仮引数表記

関数定義において、ポインタ型の仮引数の便利な表記法がある。

```
int f(int * p) { ... }
```

のような関数定義は、

```
int f(int p [ ]) { ... }
```

と書いても良い。

例

```
#include <stdio.h>
int sum (int p[ ], int size) {
    int sum=0, i;
    for (i=0; i<size; i=i+1)
        sum = sum + p[i];
    return sum;
}

int main (void) {
    int a[5] = {10,20,30,40,50};
    printf ("sum = %d\n", sum (&a[0], 5));
    return 0;
}
```

仮引数のint p [] という表記はint * pと同じ意味である。配列を受け取っているかのように見えるので、プログラムが読みやすくなる効果がある。

int p [] を、int p [5]のように書くことも許されているが、5は無視される。int p []もint p [5]もポインタ型の仮引数の別記法であり、pは配列ではない。

配列について

```
int a [5];
```

という宣言下において、aは長さ5の配列であるが、
(少数の例外を除いて)aは配列aの先頭要素への
ポインタ、すなわち&a[0]と同じ意味である。

(例外) sizeofの引数、&の引数は例外である。

```
int a[5];
```

という宣言下において、sizeof(a)は配列a全体の
サイズ(演習室では20)、&aは配列a全体へのポ
インタ(int (*) [5] 型)である。

これらの説明はしない(この講義の範囲外)。

例

```
#include <stdio.h>
int sum (int p[ ], int size) {
    int sum=0, i;
    for (i=0; i<size; i=i+1)
        sum = sum + p[i];
    return sum;
}
int main (void) {
    int a[5] = {10,20,30,40,50};
    printf ("sum = %d¥n", sum (a, 5));
    return 0;
}
```

aは、aの先頭要素へのポインタ(&a[0])の意味である。

(補足)[] 記法について

29ページで書いた通り、 $a[i]$ は $*(a + i)$ の別記法であり、 $i[a]$ は $*(i + a)$ の別記法である。

$a + i$ と $i + a$ の評価結果は同じなので、 $a[i]$ と $i[a]$ の評価結果も同じである。たとえば、

```
int a [5];
```

という宣言下において、 $a[0]$ は $0[a]$ で置き換えてもよく、 $a[1]$ は $1[a]$ で置き換えてよく、.....、 $a[4]$ は $4[a]$ で置き換えてよい。

定義上は許されているが、 $0[a]$, $1[a]$ のような書き方はプログラムの可読性を著しく低下させるので避けるべき。

printfによるデバッグ

プログラム実行時にセグメントエラー等で停止した場合、gdbなどのデバッガを使ってデバッグをするのが一般的だが、もっと原始的なやり方として、printfを何カ所かに挿入することによってどの位置で停止したかを突き止めるというやり方もある。その際、停止する場所の範囲を半分ずつ狭めていくと効率がよい。場所が特定されたら、次に何が間違っているのかを考えていくことになる。

printfでデバッグする際の注意事項を次のページに記載する。

printfによるデバッグの注意事項

printfが実行されても、すぐ画面に文字が出力され
るとは限らず、内部に文字列が溜まったままにな
る場合がある。対処法としては、printfの直後に

```
fflush(stdout);
```

を入れればよい。これにより、内部に溜まった文字
列が(あれば)画面に出てくる。

次のページに具体例を示す。

例

```
#include <stdio.h>
int main (void) {
    printf ("abc");
    while (1);
    return 0;
}
```

左のようなプログラムを実行したとき、abcという文字列が画面に表示されない場合がある。そのような場合、`fflush(stdout);`という関数呼び出しを下のプログラムのように追加すると、強制的に画面に書き出される。

```
#include <stdio.h>
int main (void) {
    printf ("abc");
    fflush(stdout);
    while (1);
    return 0;
}
```

基本課題1

第4回基本課題1のプログラムについて、成績の配列及其の長さを引数に取り、80点以上の人数を返す関数numOfGradeAを以下の形で定義し、それを呼び出す形で書き直せ。

```
int numOfGradeA (int * seiseki, int length) { ... }
```

[実行例]

```
% ./a.out < seiseki.txt
```

```
80点以上的人是81人です。
```

基本課題2

第4回基本課題2のプログラムについて、成績の配列及其の長さを引数に取り、平均点をdouble型で返す関数averageを以下の形で定義し、それを呼び出す形で書き直せ。

```
double average (int * seiseki, int length) { ... }
```

[実行例]

```
% ./a.out < seiseki.txt
```

平均点は82.70点です。

発展課題1

2つのn次元実数値ベクトルの内積を求める関数innerProdを定義せよ。ただし、n次元ベクトルは、長さnのdouble型の配列で表すものとする。関数innerProdを、配列の先頭要素へのポインタ2つと、配列の長さ(int 型)を引数にとり、結果をdouble型の値で返す関数として定義せよ。

```
double innerProd (double *a, double *b, int size) { ... }
```

main関数で2つの3次元実数値ベクトルを長さ3の配列a,bで表し、各要素をキーボードから取得し、innerProd関数を呼び出して結果を以下のように小数点以下2桁まで表示すること。

[実行例]

a[0] = 1.1

a[1] = 1.2

a[2] = 1.3

b[0] = 2.1

b[1] = 2.2

b[2] = 2.3

aとbの内積は7.94です。

(補足)関数innerProdの仮引数のdouble *a, double *bの部分はdouble a[], double b[]と書いても同じ意味である。

(補足)scanfでのdouble型の読み取りの変換指定子は%lfを使ってください。

発展課題2

int 型の配列中の最小値を求める関数minを以下の形で定義せよ。

```
int min (int *p, int size) { ... }
```

関数minは、配列の先頭要素へのポインタp, 配列の要素数sizeの2つを引数として受け取り、最小値を返す関数として定義せよ。main関数で長さ5の配列を宣言し、各要素をキーボードから取得し、min関数を呼び出して結果を以下のように表示すること。

[実行例]

長さ5の配列を入力してください。

a[0] = 3

a[1] = 6

a[2] = -5

a[3] = 7

a[4] = 2

最小値は-5です。

(補足) 関数minの仮引数 int * pの部分は int p [] と書いても同じ意味である。

(補足) int型の最大値INT_MAXを使ってよい。その場合、limits.hをインクルードする必要がある。

発展課題3

int型の配列およびその長さを引数として受け取り、配列中の要素を小さい順に並び変える関数sortを以下の形で定義せよ。

```
void sort (int *a, int size) { ... }
```

main関数で長さ5の配列を宣言し、各要素をキーボードから取得し、sort関数を呼び出して結果を以下のように表示すること。

[実行例]

長さ5の配列を入力してください。

a[0] = 10

a[1] = 50

a[2] = 38

a[3] = 80

a[4] = 60

ソート後:

a[0] = 10

a[1] = 38

a[2] = 50

a[3] = 60

a[4] = 80

発展課題4(この課題は分かる人だけでokです)

2次元配列a, b, sを使って、2*2の行列の和を計算して表示するプログラムを書け。行列の各要素の値はint型とし、キーボードから読み込むようにせよ。ただし、行列の和を計算する関数sumを

```
void sum (int a[][2], int b[][2], int s[][2]) { ... }
```

の形で定義し、main関数で各要素をキーボードから取得し、関数sumを呼び出して結果を以下のように表示すること。

[実行例] 行列aを入力してください: 行列a, bの和は

a[0][0] = 1

s[0][0] = 2

a[0][1] = 2

s[0][1] = 3

a[1][0] = 3

s[1][0] = 4

a[1][1] = 4

s[1][1] = 5

行列bを入力してください: です。

b[0][0] = 1

b[0][1] = 1

b[1][0] = 1

b[1][1] = 1

(補足)関数sumの仮引数 int a[][2]の部分は、int(*a)[2]と書いても同じ意味である。b, sも同様。